

MLE-STAR Action Plan

Next phases: stabilization, skrub integration, and MCTS

Branch: feature/reconstruct-mle-star-openai

Generated: 2026-06-17

1. What is this document?

This document is a practical action plan for the next stages of the MLE-STAR project. It is written so that a teammate who has never seen the codebase can understand what we are building, why we are building it, and exactly what tasks need to be done.

The plan covers three phases:

1. Phase 1 - Stabilization and tests
2. Phase 3 - skrub integration
3. Phase 4 - MCTS integration

These phases were chosen because they provide a solid foundation (tests), then a modern tabular-ML building block (skrub), and finally a search mechanism (MCTS) to optimize pipelines automatically.

2. Project context

2.1 What is MLE-STAR?

MLE-STAR is an autonomous multi-agent system for machine-learning engineering. In simple terms: you give it a dataset and a task description, and a team of AI agents designs, trains, debugs, and improves a machine-learning solution for you.

Think of it as automating a small data-science team:

- One agent reads the task description.
- Another proposes which models to try.
- Another writes a first training script.
- Another merges the best models.
- Another checks for data leakage.
- Another runs ablation studies to find what matters.
- Another builds an ensemble.
- A final agent writes submission.csv.

Each agent is powered by a large language model (LLM).

2.2 What have we already built?

The original MLE-STAR was built on Google's Agent Development Kit (ADK) and Gemini. Because we cannot use those services in our environment, we rebuilt the system to run on the university's OpenAI-compatible API.

The reconstruction branch (feature/reconstruct-mle-star-openai) already implements:

- Multiple model candidates and a merger.
- Ablation-based refinement.
- Ensemble agent.
- Data-usage and data-leakage checkers.
- A full end-to-end pipeline validated on California Housing.

In other words, the core multi-agent architecture is working. The next steps are to make it robust, modern, and searchable.

3. Why these three phases?

3.1 Phase 1 - Stabilization and tests

Right now the pipeline works, but it relies heavily on fallbacks to keep going when the small university model makes mistakes. Before adding more complexity, we need to:

- Make sure the existing code does not break silently.
- Document the expected behavior of each utility.
- Catch regressions early with automated tests.
- Validate that the pipeline works on more than one dataset.

Without this phase, every new feature risks destabilizing the whole system.

3.2 Phase 3 - skrub integration

skrub is a Python library for building machine-learning pipelines over messy tabular data: high-cardinality categoricals, dates, free text, and related tables. Unlike raw scikit-learn, skrub lets you declare a pipeline as a DAG of DataOps nodes. Some nodes are choices (choose_from,

choose_int, choose_float), so a single skrub plan can represent many concrete configurations. Why integrate skrub? Because the current system generates raw Python scripts from scratch. That is flexible but fragile: a small model often writes broken code. skrub provides a structured, reusable set of building blocks that the LLM can compose instead of writing every line by hand. This should make generated pipelines more robust and easier to optimize.

3.3 Phase 4 - MCTS integration

MCTS (Monte Carlo Tree Search) is a search algorithm that explores a large space of decisions by building a search tree and simulating outcomes. In our case, the decisions are the choices inside a skrub pipeline: which encoder to use, whether to scale, which model family, which hyperparameters, etc.

Why MCTS? Because the LLM alone is bad at exhaustive search. It can propose a good plan, but it cannot cheaply try thousands of configurations. MCTS can. By combining the two, the LLM proposes the structure and MCTS navigates the discrete choices to find the best concrete pipeline.

4. Existing skrub/MCTS foundation

There is already a branch called feature/mcts-skrub that contains a working MCTS engine and a skrub bridge. The most important pieces are:

- machine_learning_engineering/mcts.py: a dependency-free MCTS implementation with UCT selection.
- machine_learning_engineering/skrub_ops.py: a bridge between skrub pipelines and MCTS.
- tests/fixtures/golden_plan.py and staged_plan.py: example skrub plans for testing.
- tests/test_mcts.py, test_skrub_ops.py, test_staged.py, test_integration.py.

What is still missing in that branch:

- A_retriever: an LLM agent that converts a task description into the spec dictionary accepted by build_staged_plan.
 - A_leakage: an LLM agent that reviews staged plans for data leakage.
 - An outer orchestrator that ties the LLM, MCTS, and the existing OpenAI port into one pipeline.
- Our skrub/MCTS phases will therefore reuse most of the existing foundation and focus on integration and LLM connectors.

5. Phase 1 - Stabilization and tests

5.1 Goal

Make the current reconstruction stable, tested, and reproducible before adding skrub and MCTS.

5.2 Why this matters

- The small university model often produces malformed output; the code relies on parsers and fallbacks that must be tested.
- Without tests, a change in one agent can silently break another.
- We need baseline metrics (score, tokens, runtime) to measure future improvements.
- Adding skrub and MCTS will multiply the number of moving parts; the foundation must be solid.

5.3 Jira cards

MLE-101: Add unit tests for state-key helpers

- Test code_util.get_updated_suffix for all agent name prefixes.
- Test code_util.get_code_state_key and get_code_execution_result_state_key.
- Ensure unknown agent names raise clear errors.

MLE-102: Add unit tests for model candidate parsing

- Test strict JSON list parsing.
- Test markdown code-block fallback.

- Test fallback to default scikit-learn candidates when parsing fails.

MLE-103: Add unit tests for score parsing

- Test exact format 'Final Validation Performance: 123.45'.
- Test formatted output with extra text or units.
- Test missing performance line returns None.

MLE-104: Add unit tests for leakage checker

- Test 'No Data Leakage' response leaves code unchanged.
- Test 'Yes Data Leakage' response replaces the reported block.
- Test graceful handling of malformed JSON responses.

MLE-105: Add integration test for initialization pipeline

- Run initialization with mocked LLM responses.
- Verify state contains init_code, exec results, and train_code_0.
- Verify fallback to best individual candidate when merger fails.

MLE-106: Validate pipeline on a second dataset

- Choose a small tabular classification dataset (e.g. Titanic or a synthetic dataset).
- Add task folder under machine_learning_engineering/tasks/.
- Run the pipeline and document score, tokens, and runtime.
- Fix any dataset-specific failures.

MLE-107: Document baseline metrics

- Run the pipeline 5 times on California Housing.
- Record best score, submission success rate, average tokens, and runtime.
- Store baseline in docs/baseline.md.

6. Phase 3 - skrub integration

6.1 Goal

Allow the LLM to generate skrub-based pipelines instead of raw scikit-learn scripts. This makes generated code more structured and reusable.

6.2 Why this matters

- skrub handles messy tabular data (high-cardinality categoricals, dates, text, related tables) better than hand-written pandas code.
- A skrub plan with choices is a natural search space for MCTS.
- It reduces the amount of code the LLM has to write from scratch, which lowers the chance of syntax and logic errors.

6.3 Jira cards

MLE-201: Port skrub/MCTS foundation from feature/mcts-skrub

- Cherry-pick or merge mcts.py and skrub_ops.py into the reconstruction branch.
- Port the test fixtures (golden_plan.py, staged_plan.py).
- Port the existing tests and make them pass in the new environment.
- Update pyproject.toml if additional skrub dependencies are needed.

MLE-202: Implement A_retriever: LLM to skrub spec

- Design a JSON/YAML spec format that build_staged_plan can consume.

- Write a prompt that asks the LLM to produce a spec from task_description.
- Add a robust parser with fallback defaults (e.g. a minimal regression/classification spec).
- Store the spec in state['skrub_spec_{task_id}'].

MLE-203: Add skrub candidate to model retriever

- Extend MODEL_RETRIEVAL_INSTR to mention skrub as an option.
- When skrub is selected, generate a skrub-based training script using build_staged_plan and apply_state.
- Ensure the generated script prints 'Final Validation Performance: <score>'.

MLE-204: Implement A_leakage for staged plans

- Write a prompt that reviews a skrub spec for data leakage (especially AggTarget aggregations).
- Run the checker after A_retriever produces a spec.
- If leakage is detected, ask the LLM to refine the spec and re-check.

MLE-205: Validate skrub pipeline on California Housing

- Create a skrub plan for California Housing.
- Run the skrub candidate through the initialization pipeline.
- Compare score and robustness against the best scikit-learn solution.

7. Phase 4 - MCTS integration

7.1 Goal

Use Monte Carlo Tree Search to optimize the discrete choices inside a skrub pipeline. The LLM proposes the structure; MCTS finds the best concrete configuration.

7.2 Why this matters

- The LLM cannot afford to try thousands of hyperparameter combinations.
- MCTS is designed to explore large combinatorial spaces efficiently.
- It gives us a principled way to improve the best solution found by the agents.

7.3 Jira cards

MLE-301: Connect skrub action space to MCTS engine

- Use skrub_ops.get_action_space to build the finite action space for a staged plan.
- Use skrub_ops.make_rollout_fn to produce a reward in [0, 1].
- Run a small end-to-end MCTS search on the golden plan fixture.

MLE-302: Implement outer orchestrator

- Create a new agent/pipeline module that coordinates A_retriever, MCTS, and final script generation.
- The orchestrator should: ask LLM for spec -> run A_leakage -> run MCTS -> evaluate best state -> generate final Python script.
- Store the best skrub state and its score in AgentState.

MLE-303: Integrate MCTS path into main pipeline

- Add a config flag use_skrub_mcts (default False initially).
- When enabled, run the skrub/MCTS orchestrator alongside or after the existing initialization pipeline.
- Ensure the best skrub solution competes with the best scikit-learn solution for final submission.

MLE-304: Add MCTS-specific tests

- Test that the action space covers all choices in the golden plan.
- Test that apply_state + get_state round-trips correctly.

- Test that MCTS finds a better score than the default state on a synthetic dataset.

MLE-305: Benchmark skrub+MCTS against baseline

- Run the full pipeline with `use_skrub_mcts=True` on California Housing.
- Compare final score, runtime, and token cost against the Phase 1 baseline.
- Document results and decide whether to enable skrub+MCTS by default.

8. Roadmap and dependencies

The phases should be executed in order because each one builds on the previous:

Phase 1 (Stabilization) -> Phase 3 (skrub) -> Phase 4 (MCTS)

Dependencies:

- Phase 1 must be complete before Phase 3 because skrub integration will reuse existing state keys and debugging utilities.
- Phase 3 must be complete before Phase 4 because MCTS needs a skrub action space to search.
- MLE-201 (port foundation) can happen in parallel with Phase 1 tests if a teammate works on it separately.

Rough timeline (part-time, iterative debugging):

- Phase 1: 1-2 weeks
- Phase 3: 2-3 weeks
- Phase 4: 2-3 weeks
- Total: 5-8 weeks

9. Success criteria

- Phase 1: all new tests pass; pipeline succeeds on at least two datasets; baseline metrics are documented.
- Phase 3: LLM can generate a valid skrub spec; skrub candidate runs end-to-end; score is competitive with scikit-learn baseline.
- Phase 4: MCTS finds a configuration better than the skrub default; the orchestrator generates a final executable script; the integrated pipeline produces submission.csv.

10. Conclusion

The reconstruction branch already proves that MLE-STAR can run on the university OpenAI-compatible API. The next step is not to add more agents, but to make the existing system solid (tests), modern (skrub), and searchable (MCTS).

This plan is deliberately incremental. Each phase delivers a working milestone, and each milestone makes the next one easier. The final result will be an autonomous ML engineering system that can propose, search, and execute high-quality tabular pipelines with minimal human intervention.